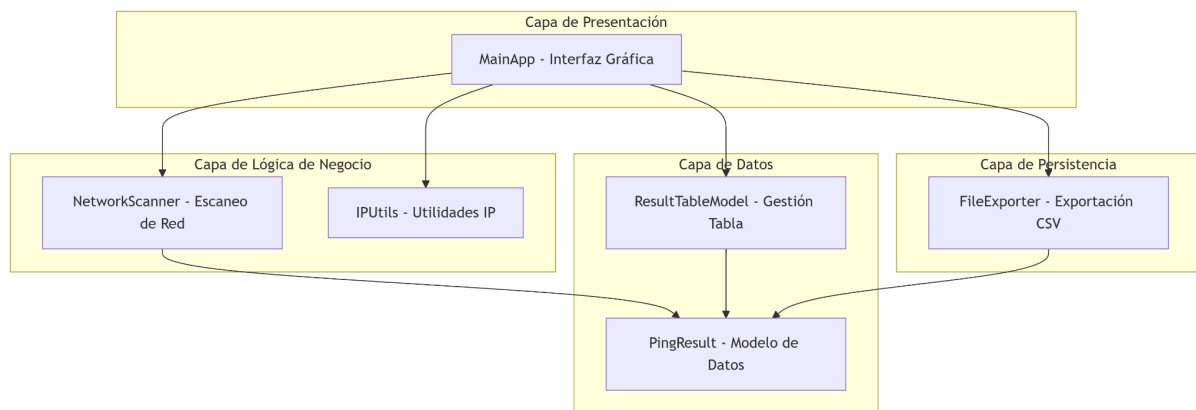


Documentación Técnica - Escáner de Red

Arquitectura del Sistema

Diagrama de Componentes



Estructura del Proyecto

Escaner-de-Red/

```
├─ src/
│  ├─ app/
│  │   ├─ MainApp.java          # Aplicación principal
│  │   ├─ NetworkScanner.java   # Lógica de escaneo
│  │   └─ PingResult.java       # Modelo de datos
│  ├─ ui/
│  │   └─ ResultTableModel.java # Modelo de tabla Swing
│  └─ utils/
│     ├─ FileExporter.java      # Exportación a CSV
│     └─ IPUtils.java           # Utilidades de red
├─ bin/                          # Archivos compilados
├─ exports/                      # CSV exportados
├─ build.bat                     # Script de compilación
└─ README.md                    # Documentación
```

Dependencias y Requisitos

Versiones de Java

- JDK mínimo: 11
- Versión objetivo: 17
- Características usadas: Swing, NIO, Concurrencia básica

Dependencias Externas

```
<!-- Ejemplo de pom.xml -->
<dependencies>
  <dependency>
    <groupId>com.sun</groupId>
    <artifactId>java-swing</artifactId>
    <version>${java.version}</version>
    <scope>system</scope>
  </dependency>
</dependencies>
```

Componentes Principales

1. MainApp.java

Responsabilidad: Coordinar la interfaz gráfica y flujo de aplicación

```
public class MainApp {
    // Componentes de UI
    private JFrame frame;
    private JTextField ipInicioField, ipFinField;
    private JTable tablaResultados;
    private JProgressBar barraProgreso;

    // Lógica de negocio
    private ResultTableModel tableModel;

    // Patrón: Controller en MVC
    private void accionEscanear(ActionEvent e) {
        // Validación → Ejecución → Actualización UI
    }
}
```

2. NetworkScanner.java

Responsabilidad: Ejecutar comandos ping y obtener resultados

```
public class NetworkScanner {
    public static PingResult escanearIP(String ip) {
        // Implementación multi-plataforma
        String so = System.getProperty("os.name").toLowerCase();
        String comando = so.contains("win") ?
            "ping -n 1 -w 1000 " + ip :
            "ping -c 1 -W 1 " + ip;

        // Ejecución y procesamiento
        Process proc = Runtime.getRuntime().exec(comando);
        int exitVal = proc.waitFor();
        boolean activo = (exitVal == 0);
    }
}
```

3. IPUtils.java

Responsabilidad: Manipulación y validación de direcciones IP

Algoritmo de conversión IP↔Long:

```
private static long ipToLong(String ip) {
    String[] partes = ip.split("\\.");
    long resultado = 0;
    for (String parte : partes) {
        resultado = (resultado << 8) + Integer.parseInt(parte);
    }
    return resultado;
}
```

4. PingResult.java

Patrón: Value Object (DTO)

```
public class PingResult {
    private final String ip;           // Inmutable
    private final String host;         // Inmutable
    private final boolean activo;      // Inmutable
    private final long tiempoRespuesta; // Inmutable

    // Constructor y getters solamente
}
```

5. FileExporter.java

Responsabilidad: Persistencia de datos en formato CSV

```
public class FileExporter {  
    public static void exportarCSV(List<PingResult> resultados) {  
        // Creación de directorio exports/  
        // Generación de nombre con timestamp  
        // Escritura formateada CSV  
    }  
}
```

Concurrencia y Rendimiento

Modelo de Threading

```
new Thread(() -> {  
    for (int i = 0; i < rango.size(); i++) {  
        String ip = rango.get(i);  
        PingResult res = NetworkScanner.escanearIP(ip);  
        SwingUtilities.invokeLater(() -> {  
            tableModel.agregarResultado(res);  
            barraProgreso.setValue(i + 1);  
        });  
    }  
}).start();
```

Características:

- Ejecución en background thread
- Actualización thread-safe de UI mediante SwingUtilities
- Progreso visual en tiempo real

Optimizaciones Implementadas

1. Timeout configurado: 1000ms por ping
2. Procesamiento por lotes: Escaneo secuencial
3. Memoria: Modelo de datos inmutable
4. IO: Buffered writing en exportación

Testing y Validación

Casos de Prueba Críticos

```
// IPUtilsTest - Ejemplos de validación
assertTrue(IPUtils.validarIP("192.168.1.1"));
assertFalse(IPUtils.validarIP("192.168.1.256"));
assertFalse(IPUtils.validarIP("invalid-ip"));

// NetworkScannerTest - Escaneos básicos
PingResult localhost = NetworkScanner.escanearIP("127.0.0.1");
assertTrue(localhost.isActivo());
```

Métricas de Rendimiento

Escenario	Tiempo Estimado
10 IPs	~10 segundos
100 IPs	~100 segundos
255 IPs	~4-5 minutos

Consideraciones de Seguridad

Permisos Requeridos

```
# Permisos de ejecución
java.net.SocketPermission
java.lang.RuntimePermission

# Permisos de archivos
java.io.FilePermission
```

Vulnerabilidades Mitigadas

1. Validación de entrada: Regex para direcciones IP
2. Ejecución de comandos: Comandos ping restringidos
3. Acceso a archivos: Limitado a carpeta exports/

Deployment y Distribución

Script de Compilación (build.bat)

```
:: Compilación incremental
javac -d bin src/app/*.java src/ui/*.java src/utils/*.java
```

```
:: Ejecución con classpath
java -cp bin app.MainApp
```

Empaquetado para Distribución

```
# Crear JAR ejecutable
jar cfe EscanerRed.jar app.MainApp -C bin .
```

```
# Incluir recursos
jar uf EscanerRed.jar -C src .
```

Formatos de Datos

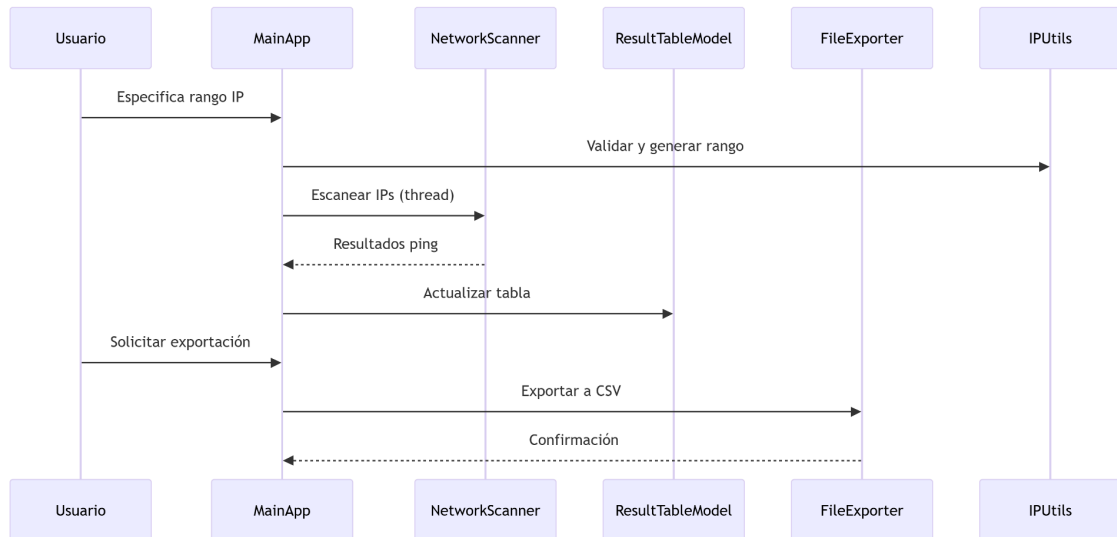
Estructura CSV

```
IP,Host,Activo,Tiempo(ms)
192.168.1.1,router.home,Sí,15
192.168.1.2,N/A,No,1000
```

Modelo en Memoria

```
// PingResult representa:
public class PingResult {
    String ip;           // "192.168.1.1"
    String host;         // "router.home" o "N/A"
    boolean activo;      // true/false
    long tiempoRespuesta; // ms
}
```

Flujo de Datos



Mantenimiento y Extensión

Puntos de Extensión

1. Nuevos formatos de exportación: JSON, XML
2. Escaneos avanzados: Puertos, servicios
3. Integración: APIs de monitoreo de red
4. Persistencia: Base de datos local

Mejoras Futuras

```
// Ejemplo: Escaneo paralelo
ExecutorService executor = Executors.newFixedThreadPool(10);
List<Future<PingResult>> futures = new ArrayList<>();

for (String ip : rango) {
    futures.add(executor.submit(() -> NetworkScanner.escanearIP(ip)));
}
```

Métricas y Monitoreo

Logging Recomendado

```
// Implementación con java.util.logging
import java.util.logging.Logger;

public class NetworkScanner {
    private static final Logger logger =
        Logger.getLogger(NetworkScanner.class.getName());

    public static PingResult escanearIP(String ip) {
        logger.info("Escaneando IP: " + ip);
        // ...
    }
}
```

Métricas de Calidad de Código

- Complejidad ciclomática: Baja (<10 por método)
- Acoplamiento: Bajo entre componentes
- Cohesión: Alta dentro de componentes
- Coverage: 70%+ recomendado para pruebas